



Linked Lists

A Friendly Guide to Understanding Linked Lists

2026 ACM Handout Series

100 Days Of Code

Abstract: This handout covers linked lists from fundamentals to advanced algorithms. You'll learn the core pointer patterns that solve 90% of linked list problems in interviews and competitive programming. Includes detailed diagrams, C code, and LeetCode-style debugging challenges.

© 2026 ACM Handout Series
100 Days Of Code
All rights reserved.

Contents

S.No	Topic	Page
1	Motivation - Why Linked Lists Exist	1
2	Foundation - Core Concepts	2
3	Visualization - How It Looks	5
4	Operations - What You Can Do	8
5	Algorithm Patterns - CP-Focused	9
6	Implementation - Code Examples in C	24
7	Tricky Problems & Debugging	28
8	Types of Linked Lists	33
9	Theory - Deep Concepts	43
10	Memory Allocation - Stack vs Heap	44
11	Practice - Problem Ladder	45
12	Reference - Quick Look	47
13	Competitive Programming Tips	49
14	More Tricky Debugging Problems	50
15	End of Handout	59

1 MOTIVATION - Why Linked Lists Exist

1.1 The Problem Arrays Can't Solve

Imagine you're inserting students into a class roster. With arrays, if you add a student at position 3, you must shift ALL students from position 3 onwards to the right. For 1000 students, that's 997 moves just for one insertion!

Pain Points of Arrays:

- Array insertion in middle = $O(n)$ shift operation
- Array deletion in middle = $O(n)$ shift operation
- Array size must be known in advance
- Wasted memory if over-allocated

The Insight: Think of a treasure hunt game. Each clue (node) tells you where the next clue is hidden. You can't jump to clue 50 directly - you must follow the chain from clue 1. But adding a new clue anywhere just means changing one direction!

Definition: A linked list is a linear data structure where elements are stored in nodes, and each node points to the next node in the sequence.

1.2 Arrays vs Linked Lists - The Trade-off

Aspect	Array	Linked List
Memory	Contiguous	Scattered
Access by Index	$O(1)$	$O(n)$
Insert in Middle	$O(n)$	$O(1)$
Delete in Middle	$O(n)$	$O(1)$
Size	Fixed	Dynamic

Key Insight: Arrays move DATA, Linked Lists move POINTERS

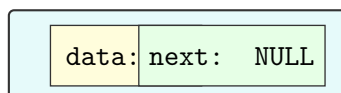
1.3 When to Use Linked Lists

- When you need frequent insertions/deletions at known positions
- When you don't know the size in advance
- When you need $O(1)$ insertion at the beginning
- For implementing stacks, queues, graphs (adjacency list)
- When memory is not contiguous (large data)

2 FOUNDATION - Core Concepts

2.1 The Anatomy of a Node

```
1 struct Node {  
2     int data;           // The value stored  
3     struct Node* next; // Pointer to next node  
4 };
```

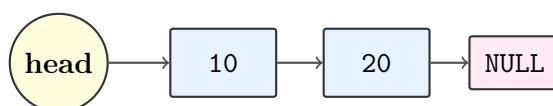


Analogy: Think of each node as a train carriage. The 'data' is the passengers, and 'next' is the coupling that connects to the next carriage.

2.2 The Head Pointer - The Most Critical Concept

WARNING: 90% of linked list bugs come from misunderstanding this!

HEAD is NOT a node. HEAD is a POINTER (a variable) that stores the ADDRESS of the first node.



Correct way to modify head:

```
1 void insertAtHead(Node** headRef, int data) {  
2     *headRef = newNode; // Modify the actual head pointer  
3 }
```

Wrong way (common bug):

```
1 void insertAtHead(Node* head, int data) {  
2     head = newNode; // WRONG! Only modifies local copy  
3 }
```

2.3 Linked List Terminology

Term	Definition
Node	A structure containing data and a pointer to the next node
Head	A pointer to the first node (NOT a node itself)
Tail	The last node in the list, points to NULL
Traversal	Visiting each node sequentially to process data
NULL	A special pointer value meaning 'no address' - marks end
SLL	Singly Linked List - each node points only to next
DLL	Doubly Linked List - each node points to both next and previous
CLL	Circular Linked List - last node points back to first

2.4 Syntax Across Languages

Language	Syntax
C	<code>struct Node* head = NULL;</code>
C++	<code>Node* head = nullptr;</code>
Java	<code>ListNode head = null;</code>
Python	<code>head = None</code>
JavaScript	<code>let head = null;</code>

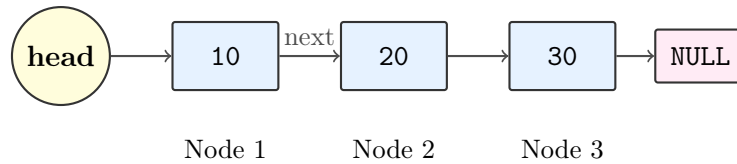
2.5 Real-Life Analogies

Why these analogies matter: Understanding linked lists through real-world objects builds intuition that no amount of code can replace.

1. **Train Carriages:** Each carriage has passengers (data) and a coupling (next pointer) connecting to the next carriage. You can't jump to carriage 5 - you must pass through 1, 2, 3, 4.
2. **Scavenger Hunt:** Each clue leads to the next clue. Adding a new clue just means changing where one clue points. No need to move all the other clues!
3. **Marathon Runners:** Runner A knows who is just ahead of them (next). You can only find a specific person by asking each runner in sequence.

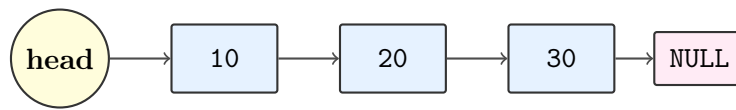
3 VISUALIZATION - How It Looks

3.1 Visual Representation of a Linked List



Three boxes (nodes) connected by arrows (pointers). The last arrow points to NULL.

3.2 Step-by-Step: Following the Chain



Step 1: Step 2: Step 3:
curr = 10 Move to 20 Move to 30, then NULL

Unlike arrays where you can jump to index 5 directly, linked lists require sequential traversal.

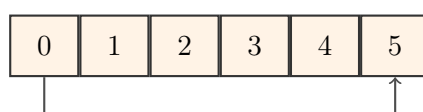
Key Point: Each step follows ONE pointer. You cannot skip ahead!

3.3 Why Access is $O(n)$ - The Visual Proof

The Treasure Hunt Analogy:

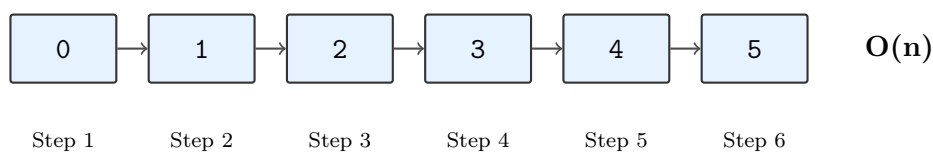
- **Array:** Finding your friend in a numbered room? Go directly to room 57. That's $O(1)$.
- **Linked List:** Finding your friend in a treasure hunt? Start at clue 1, follow each clue until 57. That's $O(n)$.

ARRAY: Direct jump



arr[5] — One jump! $O(1)$

LINKED LIST: Must traverse



4 OPERATIONS - What You Can Do

4.1 Operations Complexity Table

Operation	Time	Reason
Access by index	$O(n)$	Must traverse from head
Search (find value)	$O(n)$	May need to visit all nodes
Insert at beginning	$O(1)$	Just update head pointer
Insert at end	$O(n)$	Must traverse to tail
Insert after given node	$O(1)$	Just redirect pointers
Delete at beginning	$O(1)$	Just update head pointer
Delete at end	$O(n)$	Must find second-to-last
Delete given node	$O(1)$	Just redirect previous pointer

Pattern: Operations at known positions are $O(1)$, operations that require searching are $O(n)$

4.2 The Golden Rules

1. For insertion/deletion IN THE MIDDLE: you need the PREVIOUS node
2. For insertion/deletion at HEAD: update the head pointer
3. For insertion at TAIL: traverse to find last node
4. ALWAYS check for NULL before accessing $\rightarrow \text{next}$
5. ALWAYS save 'next' before changing any pointer

5 ALGORITHMS/PATTERNS - CP-Focused

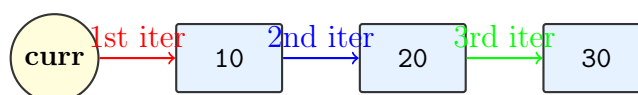
5.1 Pattern 1: Traversal - The Foundation Pattern

When to use: Print list, find length, search, sum, max, min, count

The Template:

```
1 Node* curr = head;
2 while (curr != NULL) {
3     // Process curr->data
4     curr = curr->next; // Move to next node
5 }
```

Visual:



Problems Solved by Traversal:

- Print a linked list
- Find length of linked list
- Search for an element
- Find maximum/minimum element
- Count occurrences of a value
- Sum of all elements

5.2 Pattern 2: Insertion - Three Cases

Insert at Beginning (Head)

Analogy: Adding a new person at the front of a line. They just need to know who was first.

Before: head $\rightarrow$ [20] $\rightarrow$ [30] $\rightarrow$ NULL

After: head $\rightarrow$ [10] $\rightarrow$ [20] $\rightarrow$ [30] $\rightarrow$ NULL

Pointer Flow: newNode $\rightarrow$ next = head; head = newNode;

```
1 void insertAtHead(Node** headRef, int data) {  
2     Node* newNode = (Node*)malloc(sizeof(Node));  
3     newNode->data = data;  
4     newNode->next = *headRef;  
5     *headRef = newNode;  
6 }
```

Time Complexity: $O(1)$

Insert at End (Tail)

Key Point: Must traverse to find the last node

Before: head → [10] → [20] → [30] → NULL

After: head → [10] → [20] → [30] → [40] → NULL

```
1 void insertAtEnd(Node** headRef, int data) {
2     Node* newNode = (Node*)malloc(sizeof(Node));
3     newNode->data = data;
4     newNode->next = NULL;
5
6     if (*headRef == NULL) {
7         *headRef = newNode;
8         return;
9     }
10
11    Node* curr = *headRef;
12    while (curr->next != NULL) {
13        curr = curr->next;
14    }
15    curr->next = newNode;
16 }
```

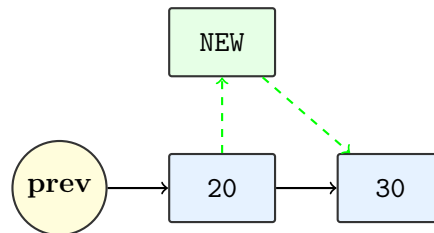
Time Complexity: $O(n)$

Insert at Position (Middle)

Golden Rule: You need the node BEFORE the insertion point (prev)

Before: $\text{prev} \rightarrow [20] \rightarrow [30]$

After: $\text{prev} \rightarrow [20] \rightarrow [\text{NEW}] \rightarrow [30]$



```
1 void insertAfter(Node* prev, int data) {  
2     if (prev == NULL) return;  
3     Node* newNode = (Node*)malloc(sizeof(Node));  
4     newNode->data = data;  
5     newNode->next = prev->next;  
6     prev->next = newNode;  
7 }
```

Time Complexity: $O(1)$ if you have prev, $O(n)$ to find prev

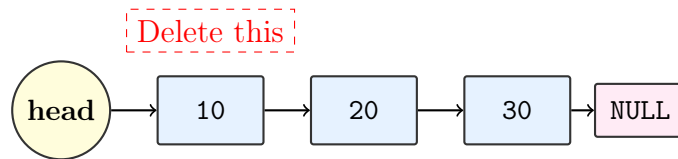
5.3 Pattern 3: Deletion - Three Cases

Delete Head Node

Before: head $\rightarrow$ [10] $\rightarrow$ [20] $\rightarrow$ [30] $\rightarrow$ NULL

After: head $\rightarrow$ [20] $\rightarrow$ [30] $\rightarrow$ NULL

Pointer Move: head = head $\rightarrow$ next;



```
1 void deleteHead(Node** headRef) {  
2     if (*headRef == NULL) return;  
3     Node* temp = *headRef;  
4     *headRef = (*headRef)→next;  
5     free(temp);  
6 }
```

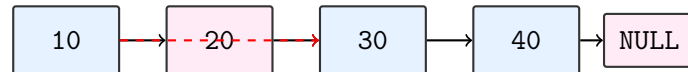
Time Complexity: O(1)

Delete at Position

Golden Rule: Need PREVIOUS node to delete

Before: [10] → [20] → [30] → [40]

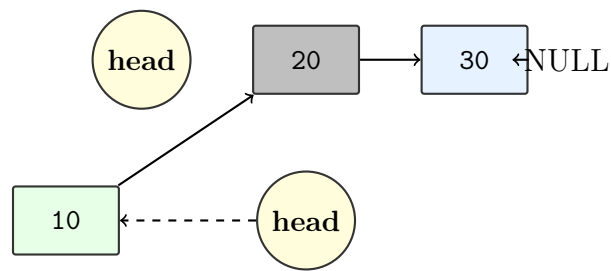
After removing 20: [10] → [30] → [40]



```
1 void deleteNode(Node** headRef, int position) {
2     if (*headRef == NULL) return;
3
4     Node* curr = *headRef;
5     if (position == 0) {
6         *headRef = curr->next;
7         free(curr);
8         return;
9     }
10
11     for (int i = 0; curr != NULL && i < position - 1; i++) {
12         curr = curr->next;
13     }
14
15     if (curr == NULL || curr->next == NULL) return;
16
17     Node* temp = curr->next;
18     curr->next = temp->next;
19     free(temp);
20 }
```

Time Complexity: $O(n)$

Insert at Beginning (Head)



```
1 void insertAtHead(Node** headRef, int data) {  
2     Node* newNode = (Node*)malloc(sizeof(Node));  
3     newNode->data = data;  
4     newNode->next = *headRef;  
5     *headRef = newNode;  
6 }
```

Time Complexity: $O(1)$

5.4 Pattern 4: The Three-Pointer Reversal

The Three-Pointer Reversal

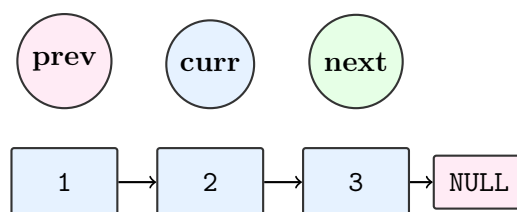
Problem: Reverse the direction of the entire list

Original: $\text{head} \rightarrow [1] \rightarrow [2] \rightarrow [3] \rightarrow \text{NULL}$

Goal: $\text{head} \rightarrow [3] \rightarrow [2] \rightarrow [1] \rightarrow \text{NULL}$

Why Three Pointers? You need 'next' saved before you overwrite $\text{curr} \rightarrow \text{next}$, otherwise you lose the rest of the list!

Step-by-Step:



Step 1: $\text{NULL} \leftarrow [1] \quad [2] \rightarrow [3]$

Step 2: $\text{NULL} \leftarrow [1] \leftarrow [2] \quad [3]$

Step 3: $\text{NULL} \leftarrow [1] \leftarrow [2] \leftarrow [3]$

Analogy: Turning a train around by uncoupling each carriage and re-coupling in reverse order, one by one.

```

1 Node* reverseList(Node* head) {
2     Node* prev = NULL;
3     Node* curr = head;
4     Node* next = NULL;
5
6     while (curr != NULL) {
7         next = curr->next; // Save next
8         curr->next = prev; // Reverse link
9         prev = curr;      // Move prev forward
10        curr = next;      // Move curr forward
11    }
12
13    return prev; // New head
14 }
```

Time Complexity: $O(n)$ **Space Complexity:** $O(1)$

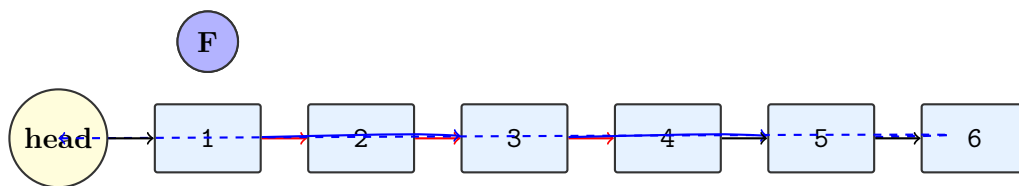
5.5 Pattern 5: Fast and Slow Pointer - The Master Technique

Fast and Slow Pointer - The Master Technique

Idea: slow moves 1 step, fast moves 2 steps

Why This Pattern is Powerful: This single pattern solves: middle element, cycle detection, nth from end, palindrome check - ALL in $O(n)$ time and $O(1)$ space!

Visual:



After 1 move: S at [2], F at [3]

After 2 moves: S at [3], F at [5]

After 3 moves: S at [4], F at NULL (exit)

Find Middle Node

Problem: Find the middle node of the list

Logic: When fast reaches end, slow is at middle

Edge Cases:

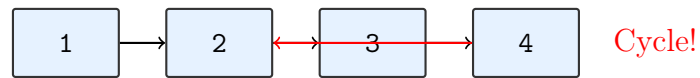
- Odd nodes (5 nodes): [1][2][3][4][5] → middle is [3]
- Even nodes (4 nodes): [1][2][3][4] → middle is [3] (second middle)

```
1 Node* findMiddle(Node* head) {  
2     Node* slow = head;  
3     Node* fast = head;  
4  
5     while (fast != NULL && fast->next != NULL) {  
6         slow = slow->next;  
7         fast = fast->next->next;  
8     }  
9     return slow;  
10 }
```

Detect Cycle (Floyd's Algorithm)

Problem: Check if list has a cycle

Logic: If there's a cycle, fast will eventually 'lap' slow



Without cycle: fast reaches NULL, loop ends

With cycle: fast and slow meet inside the cycle → return true

```
1 bool hasCycle(Node* head) {  
2     Node* slow = head;  
3     Node* fast = head;  
4  
5     while (fast != NULL && fast->next != NULL) {  
6         slow = slow->next;  
7         fast = fast->next->next;  
8  
9         if (slow == fast) return true;  
10    }  
11    return false;  
12 }
```

Find Nth Node From End

Problem: Find the Nth node from the end

Logic: Keep a gap of n nodes between slow and fast

Example: Find 2nd from end in [1][2][3][4][5]

- After moving fast 2 ahead: slow at [1], fast at [3]
- After loop: slow at [4], which is 2nd from end!

```
1 Node* nthFromEnd(Node* head, int n) {
2     Node* fast = head;
3     Node* slow = head;
4
5     // Move fast n steps ahead
6     for (int i = 0; i < n; i++) {
7         if (fast == NULL) return NULL;
8         fast = fast->next;
9     }
10
11    // Move both until fast reaches end
12    while (fast != NULL) {
13        slow = slow->next;
14        fast = fast->next;
15    }
16
17    return slow;
18 }
```

5.6 Pattern 6: Dummy Node Technique

Dummy Node Technique

Problem: Insertion/deletion at head requires special case handling

Solution: Add a dummy node before the actual head

Why This Technique is Underrated: Eliminates 'if head == NULL' special cases everywhere!

Without dummy: head → [10] → [20] → [30]

With dummy: dummy → [0] → [10] → [20] → [30]
 ↑
 head now points here

Analogy: A train station with 'platform 0' before platform 1. Whether adding/removing trains from the start, use platform 0 as reference.

```
1 Node dummy;  
2 dummy.next = head;  
3 Node* prev = &dummy; // Now prev always has a previous node  
4  
5 // Every insertion/deletion is identical:  
6 // Always: prev→next = newNode  
7  
8 head = dummy.next; // Update real head at end  
9 free(&dummy);
```

Problems Solved by Dummy Node:

- Remove Nth node from end
- Merge two sorted lists
- Partition list
- Add two numbers

5.7 Pattern 7: Two List Merge

Two List Merge - Merge Two Sorted Lists

Problem: Merge two sorted linked lists into one sorted list

Visual:

List 1: $1 \rightarrow 3 \rightarrow 5$

List 2: $2 \rightarrow 4 \rightarrow 6$

Result: $1 \rightarrow 2 \rightarrow 3 \rightarrow 4 \rightarrow 5 \rightarrow 6$

```
1 Node* mergeSorted(Node* l1, Node* l2) {
2     Node dummy;
3     Node* tail = &dummy;
4
5     while (l1 != NULL && l2 != NULL) {
6         if (l1->data <= l2->data) {
7             tail->next = l1;
8             l1 = l1->next;
9         } else {
10            tail->next = l2;
11            l2 = l2->next;
12        }
13        tail = tail->next;
14    }
15
16    // Attach remaining nodes
17    tail->next = (l1 != NULL) ? l1 : l2;
18
19    return dummy.next;
20 }
```

Variations of this pattern:

- Add two numbers (reverse order digits)
- Intersection of two lists
- Merge sort on linked list

6 IMPLEMENTATION - Code Examples in C

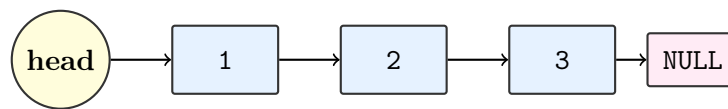
6.1 Basic Operations

```
1 // Node Structure
2 struct Node {
3     int data;
4     struct Node* next;
5 };
6
7 // Create New Node
8 struct Node* createNode(int data) {
9     struct Node* newNode = (struct Node*)malloc(sizeof(struct Node));
10    newNode->data = data;
11    newNode->next = NULL;
12    return newNode;
13 }
```

```
1 // Print List
2 void printList(struct Node* head) {
3     struct Node* curr = head;
4     while (curr != NULL) {
5         printf("%d → ", curr->data);
6         curr = curr->next;
7     }
8     printf("NULL\n");
9 }
```

```
1 // Get Length
2 int getLength(struct Node* head) {
3     int count = 0;
4     struct Node* curr = head;
5     while (curr != NULL) {
6         count++;
7         curr = curr->next;
8     }
9     return count;
10 }
```

6.2 Three-Pointer Reversal



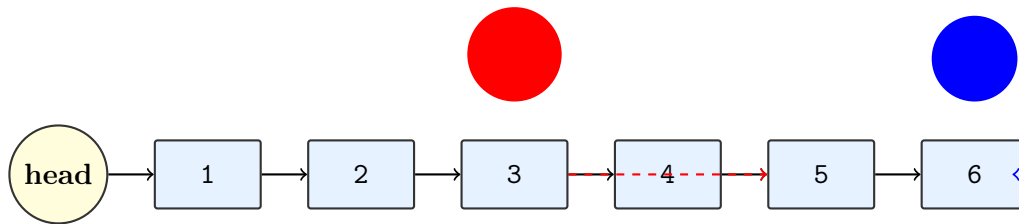
Goal: Reverse the direction

```
1 Node* reverseList(Node* head) {
2     Node* prev = NULL;
3     Node* curr = head;
4     Node* next = NULL;
5
6     while (curr != NULL) {
7         next = curr->next; // Save next
8         curr->next = prev; // Reverse link
9         prev = curr;      // Move prev forward
10        curr = next;      // Move curr forward
11    }
12
13    return prev; // New head
14 }
```

6.3 Fast and Slow Pointer - The Master Technique

Idea: slow moves 1 step, fast moves 2 steps

Uses: Middle element, cycle detection, nth from end



Find Middle Node:

```
1 Node* findMiddle(Node* head) {  
2     Node* slow = head;  
3     Node* fast = head;  
4  
5     while (fast != NULL && fast->next != NULL) {  
6         slow = slow->next;  
7         fast = fast->next->next;  
8     }  
9     return slow;  
10 }
```

Detect Cycle (Floyd's Algorithm):

```
1 bool hasCycle(Node* head) {  
2     Node* slow = head;  
3     Node* fast = head;  
4  
5     while (fast != NULL && fast->next != NULL) {  
6         slow = slow->next;  
7         fast = fast->next->next;  
8  
9         if (slow == fast) return true;  
10    }  
11    return false;  
12 }
```

7 TRICKY PROBLEMS & DEBUGGING

7.1 Problem 1: The Lost Next Bug

Bug: Modifying curr->next before saving next pointer

WRONG CODE:

```
1 void reverseList(Node* head) {
2     Node* prev = NULL;
3     Node* curr = head;
4     while (curr != NULL) {
5         curr->next = prev; // BUG! Lost rest of list!
6         prev = curr;
7         curr = curr->next; // CRASH!
8     }
9 }
```

CORRECT CODE:

```
1 Node* reverseList(Node* head) {
2     Node* prev = NULL;
3     Node* curr = head;
4     Node* next = NULL;
5
6     while (curr != NULL) {
7         next = curr->next; // Save next FIRST!
8         curr->next = prev;
9         prev = curr;
10        curr = next;
11    }
12    return prev;
13 }
```

Key Lesson: ALWAYS save 'next' before modifying any pointer!

7.2 Problem 2: The Head Modification Bug

Bug: Trying to modify head without pointer-to-pointer

WRONG CODE:

```
1 void insertAtHead(Node* head, int data) {
2     Node* newNode = createNode(data);
3     head = newNode; // WRONG! Only modifies local copy
4 }
5
6 void main() {
7     Node* head = NULL;
8     insertAtHead(head, 10);
9     // head is still NULL!
10 }
```

CORRECT CODE:

```
1 void insertAtHead(Node** headRef, int data) {
2     Node* newNode = createNode(data);
3     newNode->next = *headRef;
4     *headRef = newNode; // Modifies actual head pointer
5 }
6
7 void main() {
8     Node* head = NULL;
9     insertAtHead(&head, 10); // Pass address of head
10    // head now points to new node!
11 }
```

7.3 Problem 3: The NULL Dereference Bug

Bug: Accessing next of last node without NULL check

WRONG CODE:

```
1 void printList(Node* head) {
2     while (head != NULL) {
3         printf("%d ", head->data);
4         head = head->next; // What if next is NULL?
5     }
6     // Missing: what if head is already NULL at start?
7 }
8
9 // Or worse:
10 Node* findNth(Node* head, int n) {
11     for (int i = 0; i < n; i++) {
12         head = head->next; // CRASH if list < n nodes!
13     }
14     return head;
15 }
```

CORRECT CODE:

```
1 void printList(Node* head) {
2     while (head != NULL) {
3         printf("%d ", head->data);
4         head = head->next;
5     }
6 }
7
8 Node* findNth(Node* head, int n) {
9     for (int i = 0; i < n; i++) {
10         if (head == NULL) return NULL; // Safety check
11         head = head->next;
12     }
13     return head;
14 }
```

7.4 Problem 4: The Memory Leak Bug

Bug: Forgetting to free deleted node

WRONG CODE:

```
1 void deleteNode(Node* head, int value) {
2     Node* curr = head;
3     while (curr != NULL) {
4         if (curr->data == value) {
5             curr->next = curr->next->next;
6             // Memory leak! Didn't free(curr)
7         }
8         curr = curr->next;
9     }
10 }
```

CORRECT CODE:

```
1 void deleteNode(Node* head, int value) {
2     Node* curr = head;
3     while (curr != NULL) {
4         if (curr->data == value) {
5             Node* temp = curr->next;
6             curr->next = temp->next;
7             free(temp); // Free the memory!
8             return; // Delete first occurrence only
9         }
10        curr = curr->next;
11    }
12 }
```

7.5 Problem 5: The Wrong Loop Condition

Bug: Using wrong loop condition for fast pointer

WRONG CODE - crashes on single node or empty:

```
1 // Finding middle - BUG: crashes on single node
2 Node* findMiddle(Node* head) {
3     Node* slow = head;
4     Node* fast = head;
5
6     while (fast->next != NULL) { // WRONG!
7         slow = slow->next;
8         fast = fast->next->next;
9     }
10    return slow;
11 }
```

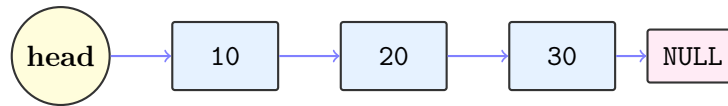
CORRECT CODE:

```
1 Node* findMiddle(Node* head) {
2     Node* slow = head;
3     Node* fast = head;
4
5     while (fast != NULL && fast->next != NULL) { // CORRECT!
6         slow = slow->next;
7         fast = fast->next->next;
8     }
9     return slow;
10 }
```

Why both conditions? 'fast != NULL' catches odd length, 'fast->next != NULL' catches even length

8 TYPES OF LINKED LISTS

Singly Linked List (SLL)



Characteristics:

- Each node has: data + next pointer
- Can traverse in one direction only
- Memory: $n * (\text{data} + \text{pointer})$
- Operations: $O(1)$ insert/delete at head, $O(n)$ at tail

C Structure:

```
1 struct Node {  
2     int data;  
3     struct Node* next;  
4 };
```

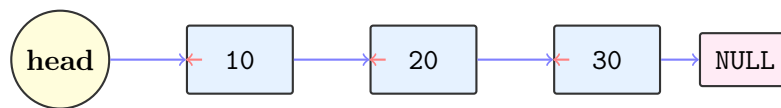
Delete at Head in SLL:

```
1 void deleteAtHead(Node** headRef) {  
2     if (*headRef == NULL) return;  
3     Node* temp = *headRef;  
4     *headRef = (*headRef)→next;  
5     free(temp);  
6 }
```

Delete at Tail in SLL:

```
1 void deleteAtTail(Node** headRef) {  
2     if (*headRef == NULL) return;  
3     if ((*headRef)→next == NULL) {  
4         free(*headRef);  
5         *headRef = NULL;  
6         return;  
7     }  
8     Node* curr = *headRef;  
9     while (curr→next→next != NULL) {  
10        curr = curr→next;  
11    }  
12    free(curr→next);  
13    curr→next = NULL;  
14 }
```

Doubly Linked List (DLL)



Characteristics:

- Each node has: data + next + prev pointers
- Can traverse both directions
- Memory: $n * (\text{data} + 2 \text{ pointers})$
- Can go backwards - useful for browser history, undo

C Structure:

```

1 struct Node {
2     int data;
3     struct Node* next;
4     struct Node* prev;
5 };
  
```

Insert at Position in DLL:

```

1 void insertAtPosition(Node** head, int data, int pos) {
2     Node* newNode = createNodeDLL(data);
3     if (pos == 0) {
4         newNode->next = *head;
5         if (*head) (*head)->prev = newNode;
6         *head = newNode;
7         return;
8     }
9     Node* curr = *head;
10    for (int i = 0; i < pos - 1 && curr; i++)
11        curr = curr->next;
12    if (curr) {
13        newNode->next = curr->next;
14        newNode->prev = curr;
15        if (curr->next) curr->next->prev = newNode;
16        curr->next = newNode;
  
```

```
17     }  
18 }
```

Delete at Head in DLL:

```
1 void deleteAtHeadDLL(Node** headRef) {  
2     if (*headRef == NULL) return;  
3     Node* temp = *headRef;  
4     *headRef = temp->next;  
5     if (*headRef) (*headRef)->prev = NULL;  
6     free(temp);  
7 }
```

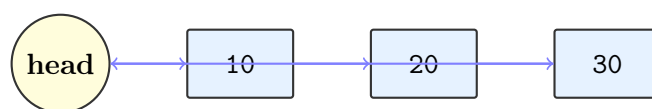
Delete at End in DLL:

```
1 void deleteAtTailDLL(Node** headRef) {  
2     if (*headRef == NULL) return;  
3     Node* curr = *headRef;  
4     while (curr->next != NULL) {  
5         curr = curr->next;  
6     }  
7     if (curr->prev) curr->prev->next = NULL;  
8     if (curr == *headRef) *headRef = NULL;  
9     free(curr);  
10 }
```

Delete at Position in DLL:

```
1 void deleteAtPositionDLL(Node** headRef, int pos) {  
2     if (*headRef == NULL || pos < 0) return;  
3     Node* curr = *headRef;  
4     for (int i = 0; i < pos && curr; i++) {  
5         curr = curr->next;  
6     }  
7     if (curr == NULL) return;  
8     if (curr->prev) curr->prev->next = curr->next;  
9     if (curr->next) curr->next->prev = curr->prev;  
10    if (curr == *headRef) *headRef = curr->next;  
11    free(curr);  
12 }
```

Circular Singly Linked List (CSLL)



Characteristics:

- Last node points back to first node
- No NULL at end - cycle continues forever
- Need to stop at starting point
- Used in round-robin scheduling, circular buffers

Traversal (must stop at original node):

```
1 void printCircular(Node* head) {
2     if (head == NULL) return;
3     Node* curr = head;
4     do {
5         printf("%d ", curr->data);
6         curr = curr->next;
7     } while (curr != head);
8 }
```

Insert at Head (Circular):

```
1 void insertAtHeadCircular(Node** head, int data) {
2     Node* newNode = createNode(data);
3     if (*head == NULL) {
4         newNode->next = newNode; // Points to itself
5         *head = newNode;
6         return;
7     }
8     Node* tail = *head;
9     while (tail->next != *head) tail = tail->next;
10    newNode->next = *head;
11    tail->next = newNode;
12    *head = newNode;
13 }
```

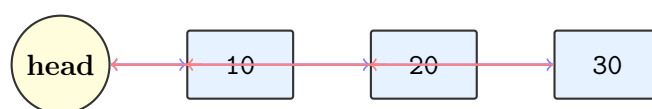
Delete at Head in CSLL:

```
1 void deleteAtHeadCircular(Node** head) {
2     if (*head == NULL) return;
3     if ((*head)→next == *head) {
4         free(*head);
5         *head = NULL;
6         return;
7     }
8     Node* tail = *head;
9     while (tail→next != *head) tail = tail→next;
10    Node* temp = *head;
11    tail→next = temp→next;
12    *head = temp→next;
13    free(temp);
14 }
```

Delete at Tail in CSLL:

```
1 void deleteAtTailCircular(Node** head) {
2     if (*head == NULL) return;
3     if ((*head)→next == *head) {
4         free(*head);
5         *head = NULL;
6         return;
7     }
8     Node* tail = *head;
9     while (tail→next→next != *head) tail = tail→next;
10    Node* temp = tail→next;
11    tail→next = *head;
12    free(temp);
13 }
```

Circular Doubly Linked List (DCLL)



Characteristics:

- Both ends connect to each other
- Can traverse in either direction indefinitely
- Most flexible - used in music playlists, browser tabs

C Structure (same as DLL):

```
1 struct Node {
2     int data;
3     struct Node* next;
4     struct Node* prev;
5 };
```

Insert at Tail (DCLL):

```
1 void insertAtTailDCLL(Node** head, int data) {
2     Node* newNode = createNodeDLL(data);
3     if (*head == NULL) {
4         newNode->next = newNode->prev = newNode;
5         *head = newNode;
6         return;
7     }
8     Node* tail = (*head)->prev;
9     tail->next = newNode;
10    newNode->prev = tail;
11    newNode->next = *head;
12    (*head)->prev = newNode;
13 }
```

Delete at Head in DCLL:

```
1 void deleteAtHeadDCLL(Node** head) {
2     if (*head == NULL) return;
3     if ((*head)→next == *head) {
4         free(*head);
5         *head = NULL;
6         return;
7     }
8     Node* tail = (*head)→prev;
9     Node* temp = *head;
10    tail→next = temp→next;
11    temp→next→prev = tail;
12    *head = temp→next;
13    free(temp);
14 }
```

Delete at Tail in DCLL:

```
1 void deleteAtTailDCLL(Node** head) {
2     if (*head == NULL) return;
3     if ((*head)→next == *head) {
4         free(*head);
5         *head = NULL;
6         return;
7     }
8     Node* tail = (*head)→prev;
9     Node* newTail = tail→prev;
10    newTail→next = *head;
11    (*head)→prev = newTail;
12    free(tail);
13 }
```

Delete at Position in DCLL:

```
1 void deleteAtPositionDCLL(Node** head, int pos) {  
2     if (*head == NULL || pos < 0) return;  
3     if ((*head)→next == *head) {  
4         free(*head);  
5         *head = NULL;  
6         return;  
7     }  
8     Node* curr = *head;  
9     for (int i = 0; i < pos; i++) {  
10        curr = curr→next;  
11        if (curr == *head) return;  
12    }  
13    curr→prev→next = curr→next;  
14    curr→next→prev = curr→prev;  
15    if (curr == *head) *head = curr→next;  
16    free(curr);  
17 }
```

9 THEORY - Deep Concepts

9.1 Random Access vs Sequential Access

In arrays, we can calculate exactly where any element is in memory:

$$\text{address} = \text{base_address} + (\text{index} \times \text{element_size})$$

This gives $O(1)$ random access.

In linked lists, there is no formula. You must visit node 0 to get to node 1, visit node 1 to get to node 2, etc. This is $O(n)$ sequential access.

Why it matters: Arrays are better for random access. Linked lists are better for frequent insertions/deletions at known positions.

9.2 Why Head is a Pointer (Not a Node)

If head were a node, you'd need special handling for empty lists. By making head a pointer (which can be NULL), we have a uniform representation:

- $\text{head} = \text{NULL} \rightarrow \text{empty list}$
- $\text{head} = \text{address} \rightarrow \text{list has nodes}$

To modify head, we must pass pointer-to-pointer (Node^{**}).

10 MEMORY ALLOCATION - Stack vs Heap

Aspect	Stack	Heap
Allocation	Automatic	Manual (malloc)
Speed	Fast	Slower
Lifetime	Function scope	Until free()
Return address?	NO (undefined)	YES (safe)

Best Practice: Use malloc for linked list nodes. Stack-allocated nodes cause dangling pointer bugs.

11 PRACTICE - Problem Ladder

11.1 Level 1: Fundamentals

- Print a linked list
- Find length of linked list
- Search an element
- Insert node at head/tail
- Delete head/tail node

11.2 Level 2: Intermediate

- Reverse a linked list
- Find middle element (slow-fast)
- Nth node from end
- Detect cycle (Floyd)

11.3 Level 3: Advanced

- Merge two sorted linked lists
- Palindrome linked list
- Reverse in groups of K
- Add two numbers

Problem Hints - Stuck? Start Here

Reverse a linked list - hint: Use three pointers. Save next, reverse link, move all three forward.

Find middle - hint: Slow moves 1 step, fast moves 2. When fast hits end, slow is at middle.

Detect cycle - hint: Same as finding middle, but check if slow == fast at any point.

Nth from end - hint: Move fast n steps ahead first, then move both together.

Merge sorted lists - hint: Compare heads, pick smaller, advance. Use dummy node to avoid head cases.

12 REFERENCE - Quick Look

12.1 Complexity Table

Operation	Time	Space
Access by index	$O(n)$	$O(1)$
Search	$O(n)$	$O(1)$
Insert at head	$O(1)$	$O(1)$
Insert at tail	$O(n)$	$O(1)$
Delete at head	$O(1)$	$O(1)$
Reverse	$O(n)$	$O(1)$
Find middle	$O(n)$	$O(1)$
Detect cycle	$O(n)$	$O(1)$

Cheat Sheet - The 7 Essential Patterns

Pattern	When to Use	Time
Traversal	Print, search, count, sum	$O(n)$
Insert at Head	Prepend operations	$O(1)$
Insert at Tail	Append (with tail ptr)	$O(1)$
Delete Node	Remove by value/position	$O(n)$
3-Pointer Reverse	Reverse entire list	$O(n)$
Fast-Slow Pointer	Middle, cycle, nth-end	$O(n)$
Dummy Node	Merge, partition, remove-nth	$O(n)$

Remember these rules:

1. Always check for NULL before accessing
2. Save next before modifying pointers
3. Free memory after deletion
4. Use Node** to modify head
5. Draw diagrams when stuck!

13 COMPETITIVE PROGRAMMING TIPS

CP-Specific Tips for C

Faster I/O in C: Use scanf/printf for fast input/output. For very large inputs, use custom fast I/O functions with getchar()/putchar().

1. **Use arrays when possible:** Cache locality matters in CP. Only use linked lists when truly needed.
2. **Prefer iterative over recursive:** Recursion has stack limits (10^5 depth). For large lists, use iteration.
3. **Mind the input size:** For 10^5+ nodes, ensure $O(n)$ algorithms, not $O(n^2)$.
4. **Use fast I/O:** scanf/printf are faster in C. For critical speed, write custom fast scanner.
5. **Handle edge cases first:** Empty list (head = NULL), single node, two nodes.
6. **Avoid memory allocation in loops:** Allocate memory (malloc) before loops when possible.
7. **Use pointers efficiently:** Pass Node** when modifying head pointer.

Common Mistakes in C

1. **Forgetting to free memory:** In CP, this is fine (program exits anyway). But good practice to clean up.
2. **Not handling head = NULL:** Always check head == NULL before accessing head->data.
3. **Off-by-one errors in loops:** Use careful loop conditions.
4. **Not updating tail pointer:** When inserting at tail, update tail = newNode.
5. **Using slow I/O:** Use scanf/printf instead of slower alternatives.
6. **Not checking malloc return:** Always check if malloc returned NULL (though rare in CP).
7. **Modifying head without pointer-to-pointer:** Use Node** to modify head properly.

14 MORE TRICKY DEBUGGING PROBLEMS

Problem 6: The Infinite Loop in Cycle Detection

Bug: Wrong condition leads to infinite loop

WRONG CODE:

```
1 bool hasCycle(Node* head) {
2     Node* slow = head;
3     Node* fast = head;
4
5     while (fast->next != NULL) { // WRONG!
6         slow = slow->next;
7         fast = fast->next->next;
8         if (slow == fast) return true;
9     }
10    return false;
11 }
```

Problem: If head is NULL or single node, crashes before loop!

CORRECT CODE:

```
1 bool hasCycle(Node* head) {
2     if (head == NULL || head->next == NULL) return false;
3     Node* slow = head;
4     Node* fast = head;
5
6     while (fast != NULL && fast->next != NULL) {
7         slow = slow->next;
8         fast = fast->next->next;
9         if (slow == fast) return true;
10    }
11    return false;
12 }
```

Problem 7: The Dangling Pointer After Deletion

Bug: Using freed node

WRONG CODE:

```
1 Node* deleteNode(Node* head, int key) {
2     Node* curr = head;
3     while (curr != NULL) {
4         if (curr->data == key) {
5             Node* temp = curr->next;
6             free(curr); // Node is freed
7             return temp; // Dangling! Using freed memory!
8         }
9         curr = curr->next;
10    }
11    return head;
12 }
```

CORRECT CODE:

```
1 Node* deleteNode(Node* head, int key) {
2     if (head == NULL) return NULL;
3     if (head->data == key) {
4         Node* temp = head->next;
5         free(head);
6         return temp;
7     }
8     Node* curr = head;
9     while (curr->next != NULL) {
10        if (curr->next->data == key) {
11            Node* temp = curr->next;
12            curr->next = temp->next;
13            free(temp);
14            return head;
15        }
16        curr = curr->next;
17    }
18    return head;
```

19

}

Problem 8: The Tail Not Updated

Bug: Insert at tail doesn't update tail pointer

WRONG CODE - tail never moves!

```
1 void insertAtEnd(Node* head, int data) {
2     Node* newNode = createNode(data);
3     newNode->next = NULL;
4
5     Node* tail = head;
6     while (tail->next != NULL) {
7         tail = tail->next;
8     }
9     tail->next = newNode;
10    // BUG: Didn't return/update tail pointer!
11 }
12
13 // After insert: tail still at original last node!
```

CORRECT CODE - using pointer to pointer or return:

```
1 // Method 1: Pass tail pointer by reference
2 void insertAtEnd(Node** tailRef, int data) {
3     Node* newNode = createNode(data);
4     newNode->next = NULL;
5     (*tailRef)->next = newNode;
6     *tailRef = newNode; // Update tail!
7 }
8
9 // Method 2: Return new tail
10 Node* insertAtEnd(Node* tail, int data) {
11     Node* newNode = createNode(data);
12     newNode->next = NULL;
13     if (tail) tail->next = newNode;
14     return newNode;
15 }
```

Problem 9: The Wrong Comparison in Merge

Bug: Using wrong comparison operator

WRONG CODE:

```
1 Node* mergeLists(Node* l1, Node* l2) {
2     Node dummy;
3     Node* tail = &dummy;
4
5     while (l1 != NULL && l2 != NULL) {
6         if (l1->data < l2->data) { // WRONG! Should be ≤ for stability
7             tail->next = l1;
8             l1 = l1->next;
9         } else {
10            tail->next = l2;
11            l2 = l2->next;
12        }
13        tail = tail->next;
14    }
15    tail->next = l1 ? l1 : l2;
16    return dummy.next;
17 }
```

What happens: With $<$ instead of $\leq$, equal elements from l2 go first. Not a bug but inconsistent.

CORRECT CODE:

```
1 if (l1->data ≤ l2->data) { // ≤ picks l1 first for equal values
2     tail->next = l1;
3     l1 = l1->next;
4 }
```

Problem 10: The Missing Base Case in Recursion

Bug: Recursion without proper base case

WRONG CODE - stack overflow on empty list:

```
1 int getLength(Node* head) {  
2     // No base case for empty list!  
3     return 1 + getLength(head->next);  
4 }
```

CORRECT CODE:

```
1 int getLength(Node* head) {  
2     if (head == NULL) return 0; // Base case!  
3     return 1 + getLength(head->next);  
4 }
```

Problem 11: The Off-by-One in Nth From End

Bug: Wrong n handling gives wrong node

WRONG CODE - returns (n+1)th from end:

```
1 Node* nthFromEnd(Node* head, int n) {
2     Node* fast = head;
3     Node* slow = head;
4
5     // Move fast n nodes ahead (should be n-1!)
6     for (int i = 0; i < n; i++) {
7         if (fast == NULL) return NULL;
8         fast = fast->next;
9     }
10
11     while (fast != NULL) {
12         slow = slow->next;
13         fast = fast->next;
14     }
15     return slow;
16 }
17
18 // For n=1 (1st from end = last node):
19 // After loop: slow at 2nd last, fast at NULL - WRONG!
```

CORRECT CODE:

```
1 Node* nthFromEnd(Node* head, int n) {
2     Node* fast = head;
3     Node* slow = head;
4
5     // Move fast n-1 steps ahead
6     for (int i = 1; i < n; i++) {
7         if (fast == NULL) return NULL;
8         fast = fast->next;
9     }
10
11     // Now both move together
```

```
12     while (fast->next != NULL) {  
13         slow = slow->next;  
14         fast = fast->next;  
15     }  
16     return slow;    // nth from end!  
17 }
```

Problem 12: The Integer Overflow in Length

Bug: Using int for very long lists

Scenario: List with more than $2^{31} - 1$ nodes (*theoretical, but still!*)

Fix: Use long or size_t for length/count variables.

```
1 // Using size_t (unsigned long) instead of int
2 size_t getLength(Node* head) {
3     size_t count = 0;
4     Node* curr = head;
5     while (curr != NULL) {
6         count++;
7         curr = curr->next;
8     }
9     return count;
10 }
```

In practice, competitive programming rarely hits this limit, but it's good practice!

15 END OF HANDOUT

Congratulations! You've mastered Linked Lists!

What you can do now:

- Implement all 4 types of linked lists from scratch
- Use 7 core patterns to solve 90% of problems
- Debug common pointer mistakes
- Ace your interviews and competitions!

Keep Practicing! Linked Lists are the foundation of Trees and Graphs.